

Algorytmy Metaheurystyczne – Lab 4–6

Andrii Hermak 284013

1. Opis zadania

Zadanie polegało na zaimplementowaniu lokalnego przeszukiwania (Local Search) dla problemu komiwojażera (TSP) z trzema operatorami sąsiedztwa:

- **Zadanie 1 – Invert (2-opt):** odwrócenie fragmentu trasy między pozycjami i i j . Pełne sąsiedztwo z listami kandydatów.
 - **Zadanie 2 – Random Invert:** w każdej iteracji losuje się n par (i, j) i wybiera najlepszą.
 - **Zadanie 3 – Transpose:** zamiana miejscami dwóch sąsiednich miast. Pełne sąsiedztwo z listami kandydatów.
-

2. Implementacja

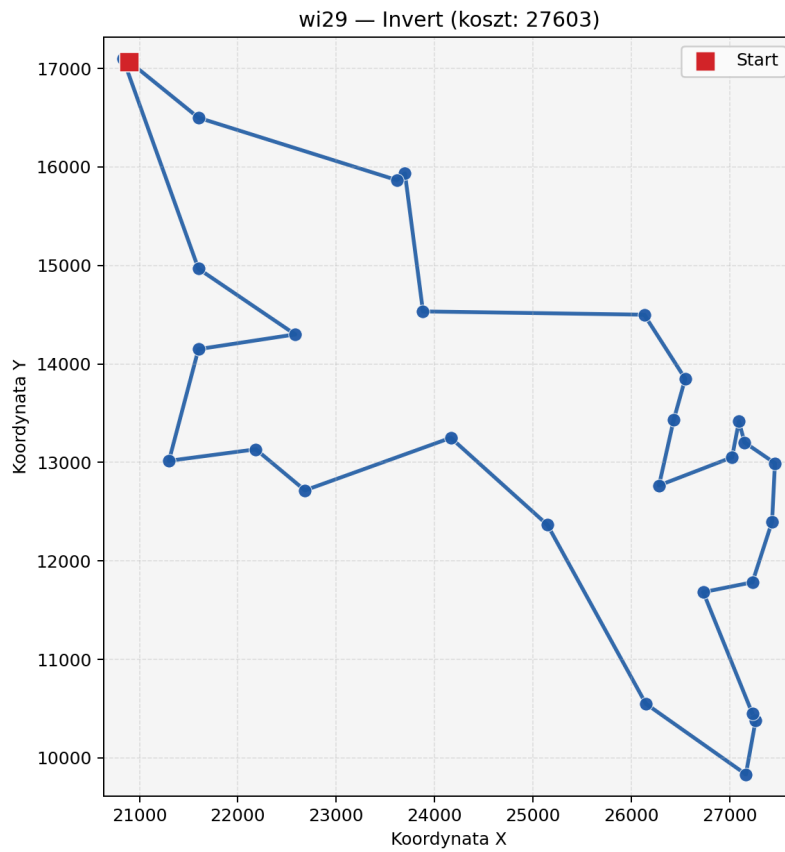
Program napisany w Pythonie z następującymi optymalizacjami:

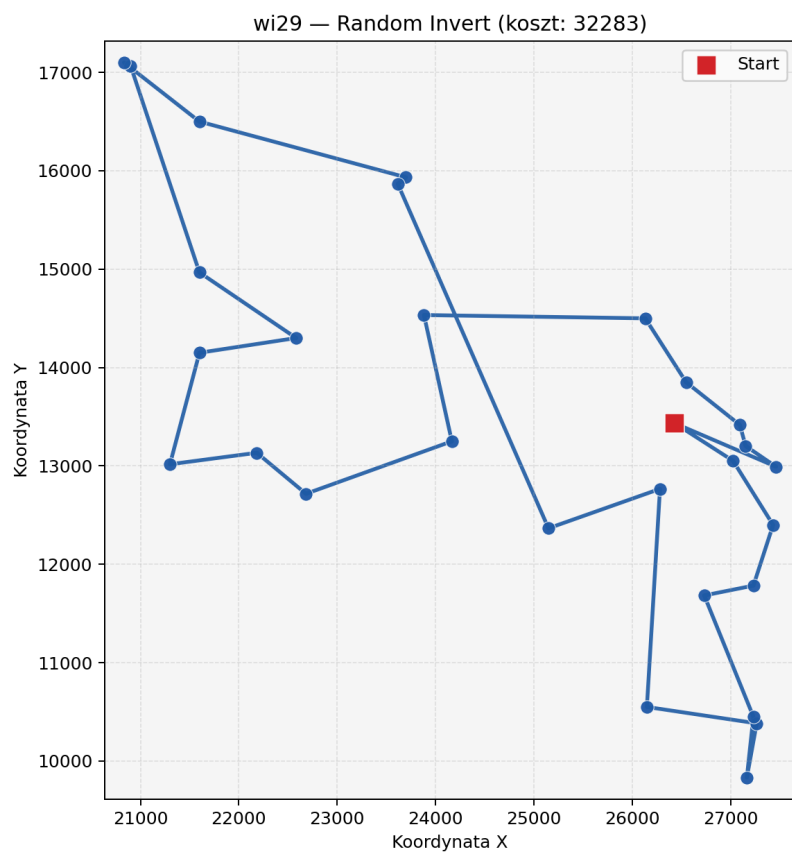
Technika	Opis
NumPy	Wektorowe obliczanie macierzy odległości (int32)
Listy kandydatów	$k = \min(n-1, 200)$ najbliższych sąsiadów na miasto
Delta $O(1)$	Zmiana kosztu bez przeliczania całej trasy
Numba JIT	<code>@njit</code> kompiluje pętle do kodu maszynowego
Numba prange	Wielowątkowe uruchamianie startów równoległe

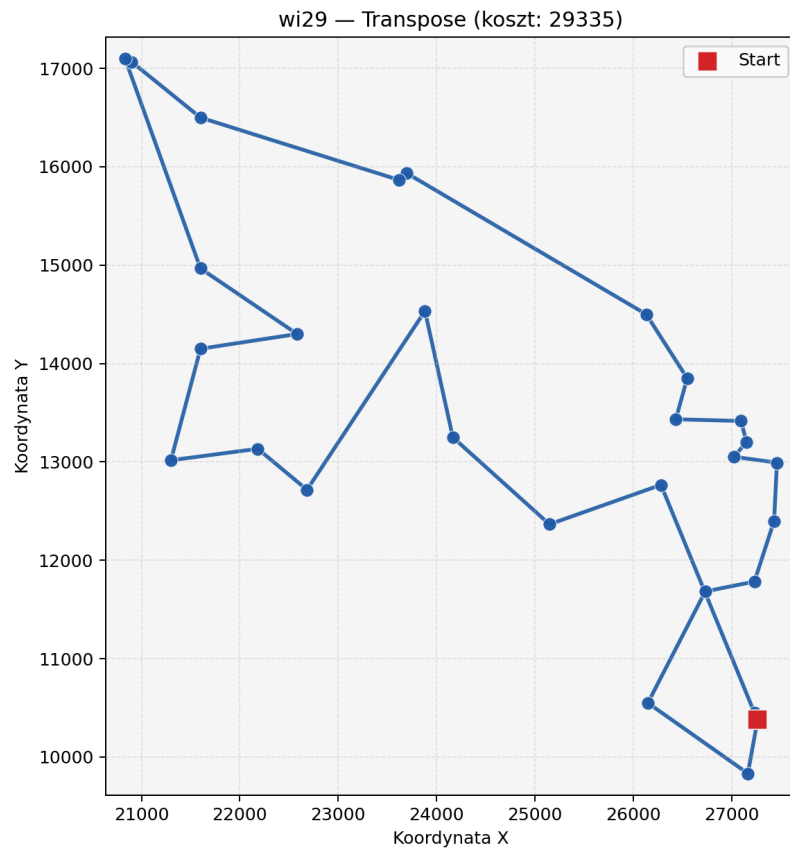
3. Wyniki

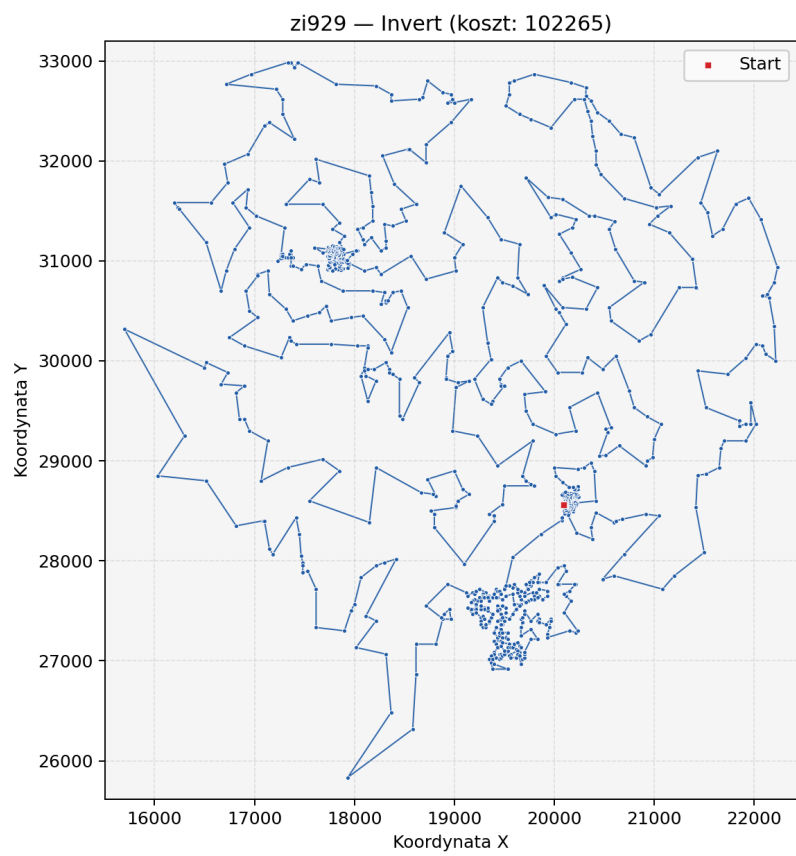
Plik	n	Z1 Best	Z1 Avg	Z1 Czas	Z2 Best	Z2 Avg	Z2 Czas	Z3 Best	Z3 Avg	Z3 Czas
wi29	29	27 603	29 193	0.00 s	32 283	40 982	0.00 s	29 335	39 018	0.00 s
dj38	38	6 656	7 081	0.00 s	9 232	10 608	0.00 s	8 503	10 462	0.00 s
qa194	194	9 843	10 338	0.04 s	13 416	15 866	0.01 s	18 214	21 990	0.11 s
uy734	734	86 639	88 578	0.56 s	145 809	167 043	0.09 s	272 952	301 052	2.16 s
zi929	929	102 265	105 358	1.13 s	138 183	150 916	0.10 s	372 236	424 813	4.14 s
mu1979	1979	93 047	95 449	3.70 s	153 470	176 961	0.41 s	840 031	971 373	27.8 6 s
ca4663	4663	1 421 477	1 443 293	43.5 s	2 485 734	2 697 866	7.3 s	14 767 504	15 827 416	760 s
tz6117	6117	437 664	443 353	82.1 s	775 898	855 307	11.2 s	4 225 067	4 406 148	1271 s
eg7146	7146	188 403	191 269	125 s	328 100	357 362	15.1 s	1 978 416	2 090 136	1871 s
ei8246	8246	230 922	233 176	187 s	475 699	532 653	22.7 s	1 944 956	2 011 380	2785 s

4. Wizualizacja

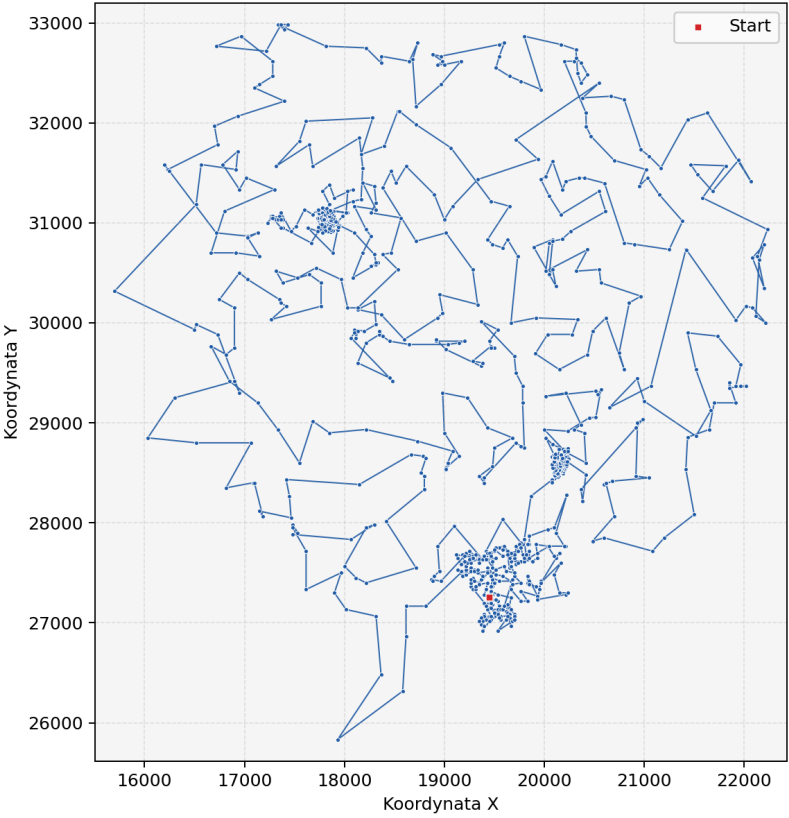


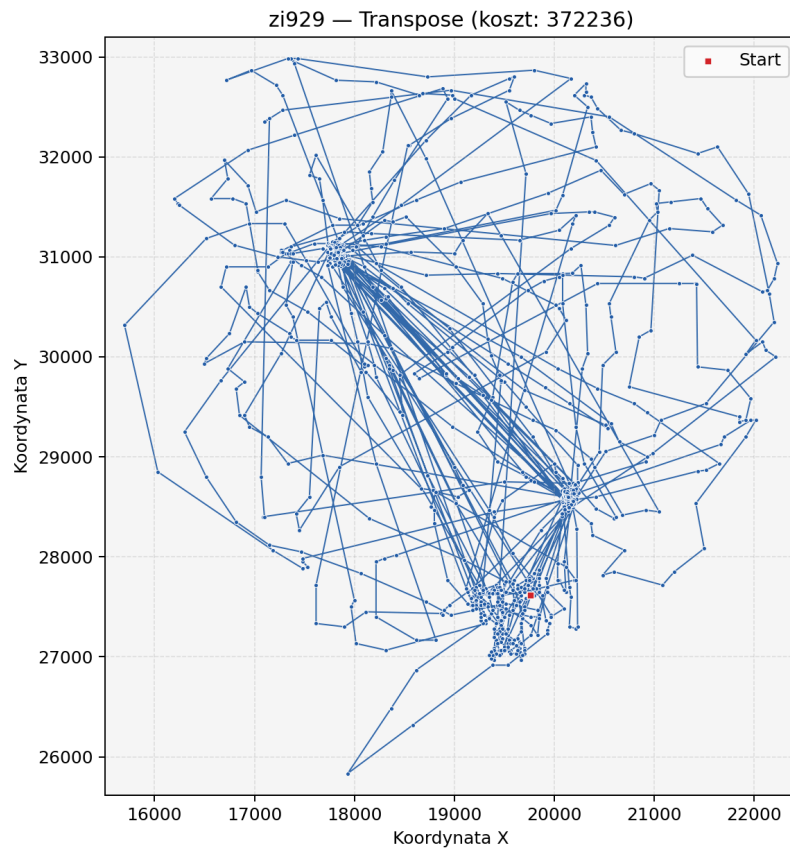




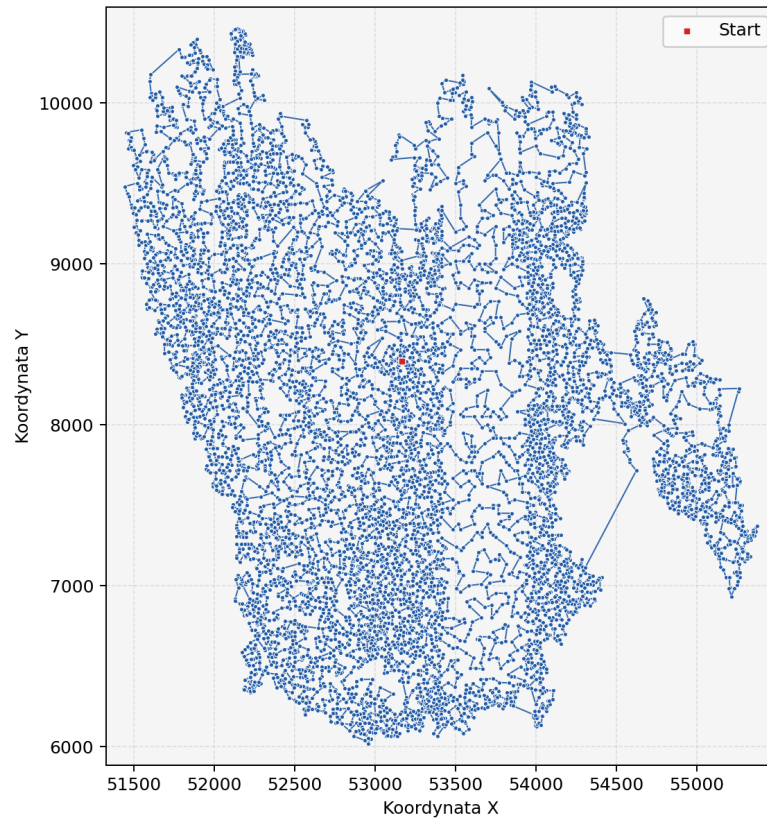


zi929 — Random Invert (koszt: 138183)

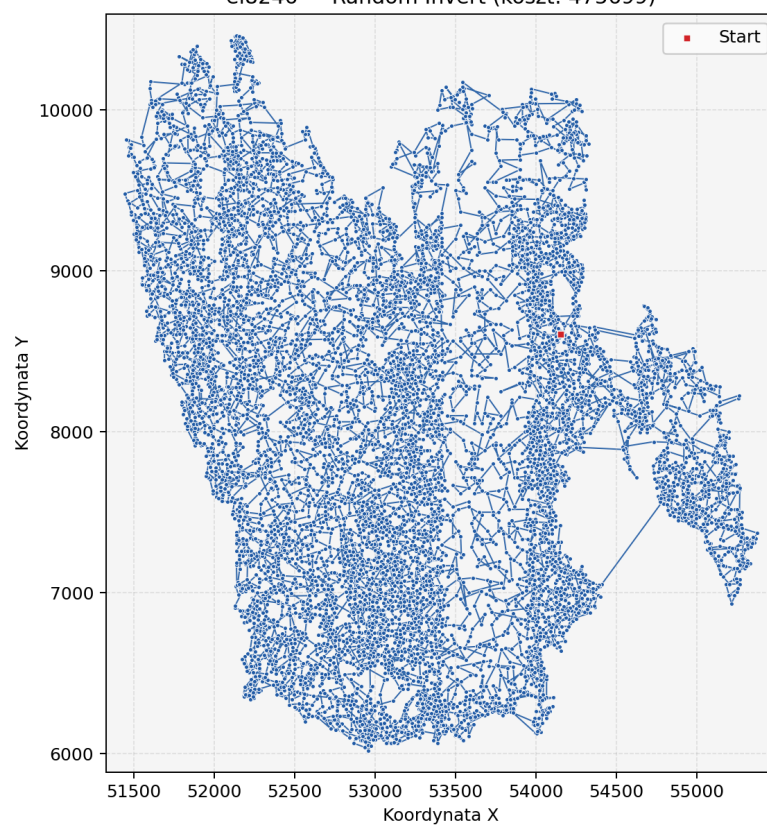


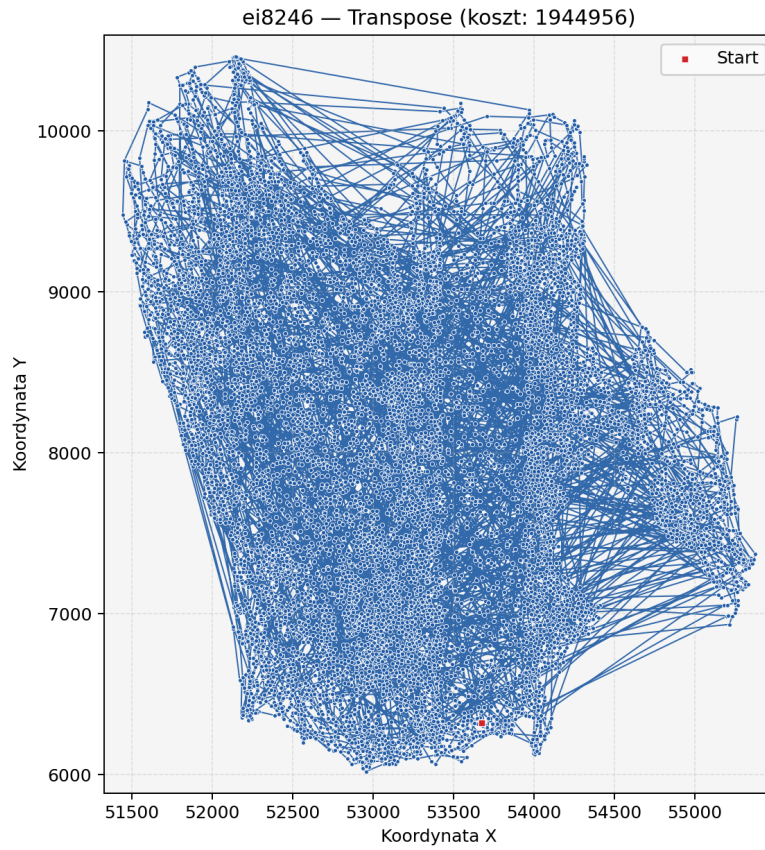


ei8246 — Invert (koszt: 230922)



ei8246 — Random Invert (koszt: 475699)





5. Analiza

Porównanie metod

Invert (2-opt) jest zdecydowanie najlepszy jakościowo. Operacja odwrócenia fragmentu trasy eliminuje przecięcia krawędzi – na małych instancjach (wi29, dj38) trasy są prawie optymalne. Wadą jest czas: dla dużych instancji (ei8246) to ~187 s.

Random Invert jest ~8–10× szybszy od pełnego Invert, ale jakość spada o ok. 50–100%. Losowe próbkowanie rzadko trafia na globalnie najlepszą parę, więc algorytm utyka w słabszych optimach lokalnych.

Transpose wypada najgorzej pod każdym względem – zarówno jakość jak i czas są najgorsze. Zamiana tylko dwóch sąsiednich miast to bardzo mała zmiana w trasie, przez co algorytm potrzebuje wielokrotnie więcej kroków i wciąż nie dochodzi do dobrego rozwiązania.

Liczba kroków

Liczba kroków rośnie proporcjonalnie do rozmiaru instancji. Transpose potrzebuje 2–3× więcej kroków niż Invert, bo pojedyncza operacja robi mniej "pracy" – wymaga więcej iteracji żeby poprawić trasę.

Wpływ list kandydatów

Dla instancji powyżej ~200 miast używane jest $k = 200$ zamiast pełnego sąsiedztwa. Dzięki temu algorytm jest wielokrotnie szybszy, ale na wizualizacjach dużych instancji widać sporadyczne przecięcia krawędzi – wynika to z tego, że optymalna para (i, j) może leżeć poza listą 200 kandydatów.

6. Wnioski

1. **Invert (2-opt)** daje najlepsze trasy i jest zalecaną metodą dla instancji do ~1000 miast.
2. **Random Invert** to dobry kompromis gdy liczy się czas – kilkakrotnie szybszy przy akceptowalnej jakości.
3. **Transpose** jest najslabszy – operator jest zbyt lokalny żeby efektywnie eksplorować przestrzeń rozwiązań.